Waste Classification for Recycling Using CNN and Transfer Learning

PRESENTED BY

Keerthana G — 2520090132

Sutte Alekhya — 2520090126

GUIDE

Dr. Y. Lakshmi Prasanna

Assistant Professor, CS-IT

Introduction

Recycling only works when waste is sorted right

Background

Urbanisation and changing consumption have increased the volume and variety of municipal solid waste. Separation is still mostly done by hand.

The domain

Computer vision lets CNNs learn edges, shapes and textures from images. Transfer learning reuses models trained on huge image sets for small datasets.

Why it is needed

Manual sorting is slow and error-prone, so recyclable material often ends up in landfill. A one-click tool makes sorting faster and consistent.

Problem statement

Mixed waste is sorted by hand

Slow and costly

Manual segregation is labour-intensive and time-consuming.

Error-prone

Different workers decide differently about the same item.

Lost recyclables

Plastic, paper, glass and metal mixed with organic waste are harder to recover.

WHO IS AFFECTED

Residential communities, recycling centres and their workers, municipal bodies and the environment.

Motivation

Why we chose this project

Our reason

It covers the full path from data to a usable app: a deep learning model, a REST API and a web front end, solving a visible everyday problem.

Real-world need

A low-cost tool anyone can use from a phone or computer: photograph an item and be told what type of waste it is.

Research motivation

TrashNet-style datasets made waste-image classification a known benchmark, and light pre-trained networks make deployment practical.

Objectives

What the project sets out to do

Main objective: identify the category of a waste item from a single photograph using a CNN with transfer learning, served through a web app.

1. Build a data pipeline for 7 classes: resize, normalise, augment, split.
2. Train and compare MobileNetV2, ResNet50 and EfficientNetB0.
3. Evaluate with accuracy, precision, recall, F1-score and confusion matrix.
4. Expose the trained model through a Flask REST API (/predict).
5. Build a web UI that uploads, previews and shows class and confidence.

Existing system

Current approach

Workers recognise the material of every item and place it in the right bin, at the source or at the sorting facility.

Technologies

Almost none in small communities and recycling centres. Automated sorting exists only in large industrial plants and is costly.

Limitations

- Slow and labour-intensive
- Inconsistent decisions
- Contaminated recyclables go to landfill
- No easy way for the public to check an item

Proposed system

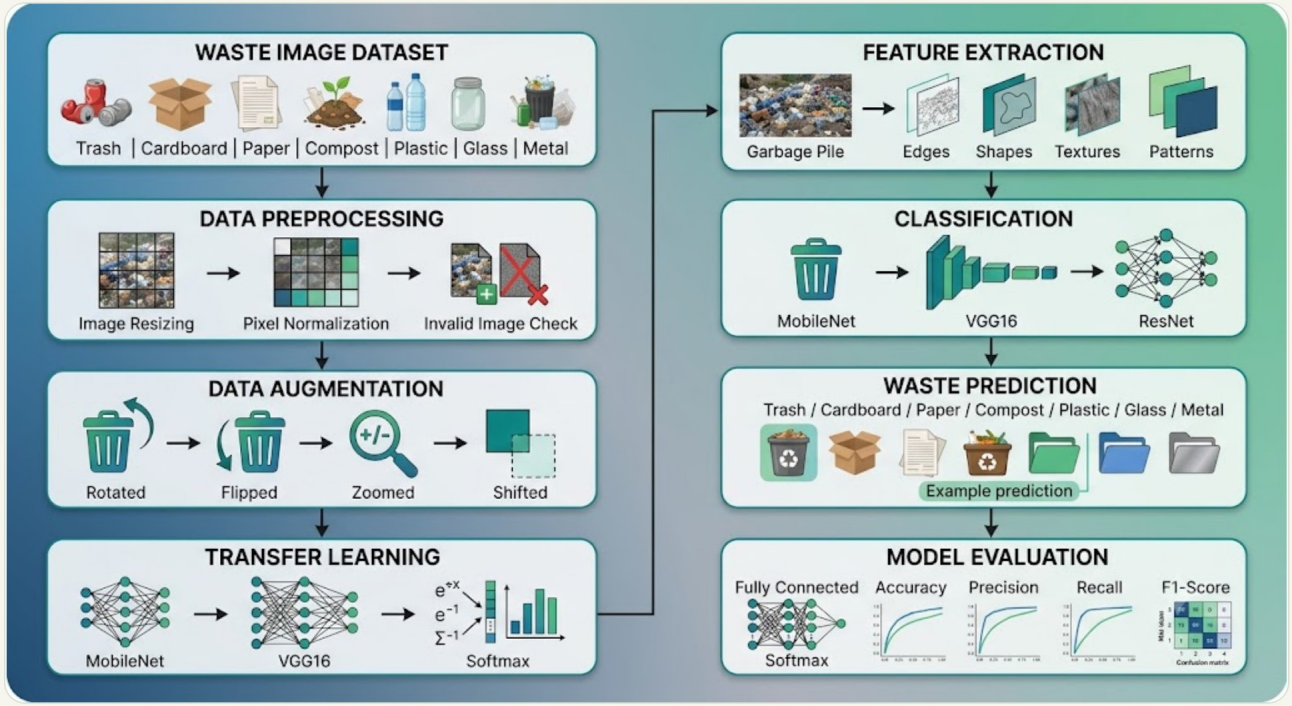
Upload a photo. A Flask API preprocesses it and a transfer-learning CNN returns the class and a confidence score.

Slow, costly → prediction in seconds

Inconsistent → one model, one standard

Lost recyclables → faster identification

Hard to access → works in any browser



System architecture

INPUT

Photo of one waste item uploaded in the browser

PROCESSING / MODULES

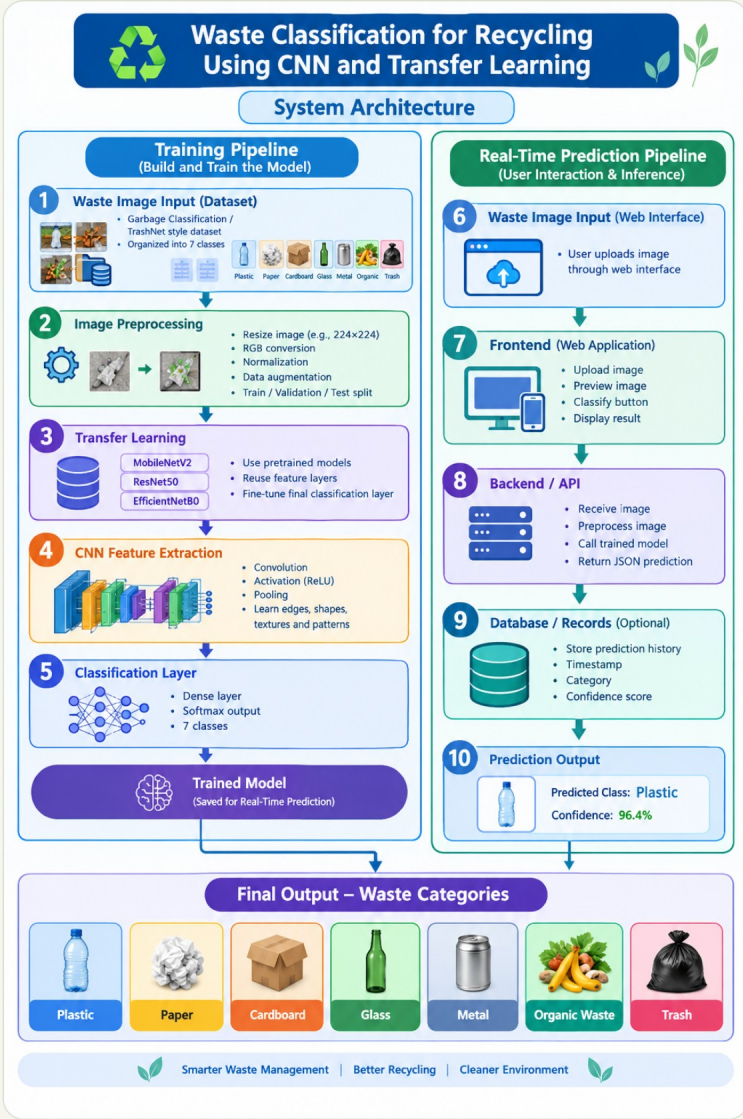
Resize 224×224, RGB, normalise → Flask API → optional SQLite log

ALGORITHMS / MODELS

CNN + transfer learning: MobileNetV2, ResNet50, EfficientNetB0

OUTPUT

One of 7 classes with a confidence score, e.g. Plastic, 96.4%



Methodology

Nine steps from data to web app

01
Collect the dataset in 7 class folders

02
Run EDA: counts, bad files, sizes, samples

03
Preprocess and split into train, val, test

04
Augment: flip, rotate, zoom, shift

05
Attach a 7-class head to a pre-trained backbone

06
Train the head, then fine-tune the top layers

07
Evaluate on the unseen test set

08
Save the model and serve it with Flask

09
Connect the web front end

End-to-end workflow

What happens when a user clicks Classify

1 · User
Selects an image

2 · Frontend
Previews, then sends it to the API

3 · Backend
Resize 224×224, RGB, normalise

4 · CNN model
Predicts the class

5 · Frontend
Shows class and confidence

REQUEST

```
POST /predict
file: waste_image.jpg
```

RESPONSE · 200 OK

```
{"class": "Plastic",
"confidence": 0.964}
```

Technology stack

Tools used at each layer

Layer	Technologies
Languages	Python · HTML5 · CSS3 · JavaScript
Frameworks	Flask REST API · TensorFlow / Keras
Libraries	Pillow · NumPy · scikit-learn · Matplotlib · Fetch API
ML models	MobileNetV2 · ResNet50 · EfficientNetB0 (.keras / .h5)
Database	SQLite (optional): timestamp, category, confidence
Hardware	Standard CPU server, no special hardware needed

System modules

Eight components working together

User interface
Upload, preview, classify, result

Data collection
TrashNet-style dataset, 7 folders

Preprocessing
Resize, normalise, augment, split

ML module
Transfer-learning CNN, 7-class softmax

Prediction module
Flask /predict returns JSON

Database
Optional SQLite history

Visualisation
Class chart, curves, confusion matrix

Security
No login; no personal data stored

Dataset

Garbage Classification / TrashNet-style data

RECORDS

[__ images in total, __ per class]

PREPROCESSING

Remove bad files · resize 224×224 · RGB · normalise · augment · train / val / test split

EDA FINDINGS

Trash is likely the smallest class; paper vs cardboard and glass vs plastic look alike.

Class	Typical items
Plastic	Bottles, containers, wrappers
Paper	Sheets, newspapers, crumpled paper
Cardboard	Boxes and packaging
Glass	Bottles and jars
Metal	Cans, tins, metal pieces
Organic Waste	Food and vegetable waste
Trash	Non-recyclable general waste

Algorithm

CNN with transfer learning

Input 224×224×3 | Augment + preprocess | **Pre-trained backbone** | Global avg pooling | Dropout 0.3 | Dense 128, ReLU | **Softmax, 7 classes**

Why a CNN

Learns edges, shapes and textures automatically, so no manual feature engineering.

Why transfer learning

Reuses ImageNet features: less data, shorter training, better accuracy on a small set.

Why three backbones

The final model is picked from test results, not assumption.

Model comparison

Three pre-trained backbones and key parameters

Model	Key idea	Parameters
MobileNetV2	Depth-wise separable convolutions; light and fast	≈ 3.4 M
ResNet50	50 layers with skip connections	≈ 25.6 M
EfficientNetB0	Compound scaling of depth, width, resolution	≈ 5.3 M

Stage 1
Adam 1e-3, 15 epochs, base frozen

Stage 2
Adam 1e-5, 10 epochs, last 30 layers

Settings
Batch 32, dropout 0.3, early stopping

Implementation

Key steps in code

Data
Keras image_dataset_from_directory, 70 / 15 / 15 split

Model
Frozen backbone plus a new 7-class head

Training
Head first, then fine-tune the top layers

Serving
Flask loads the model; JavaScript calls it with fetch()

MODEL BUILD (KERAS)

```
x = augment(inputs)
x = preprocess(x)
x = base(x, training=False)
x = GlobalAveragePooling2D()(x)
x = Dropout(0.3)(x)
x = Dense(128, activation="relu")(x)
out = Dense(7, activation="softmax")(x)
```

Results

Test-set performance

Model	Accuracy	Precision	Recall	F1-score
MobileNetV2	[__]	[__]	[__]	[__]
ResNet50	[__]	[__]	[__]	[__]
EfficientNetB0	[__]	[__]	[__]	[__]

Add chart
Training vs validation accuracy and loss

Add chart
Confusion matrix on the test set

Add chart
Per-class precision and recall

Best model: [state which of the three performed best]

Comparison

Manual sorting vs the CNN system

Aspect	Manual (existing)	CNN system (proposed)
Speed	Slow, item by item	Seconds per image
Consistency	Varies between workers	One model, plus a confidence score
Cost	Labour-intensive	Any browser, no special hardware
Access	Trained staff only	Anyone with a photo
Accuracy	[not measured]	[best-model test accuracy]